

Outline

- Introduction
- **Data Manipulation Language (DML)**
 - Simple SQL Queries
 - Nested SQL Queries
 - **Aggregation and Grouping**
 - Others, More Example: Relational Algebra and SQL
- **Advanced Concepts (DDL)**
 - Introduction to View
 - Introduction to Assertion/Trigger
 - Introduction to Database Programming

Aggregate Operators

- SQL allows the use of arithmetic expressions.
- SQL supports five aggregate operations, which can be applied on any column of a relation.

COUNT([DISTINCT] A)	The number of (unique) values in the A column.
SUM ([DISTINCT] A)	The sum of all (unique) values in the A column.
AVG ([DISTINCT A)	The average of all (unique) values in the A column.
MAX (A)	The maximum value in the A column.
MIN (A)	The minimum value in the A column.

Examples

sailors	<u>sid</u>	sname	rating	age
Boats	<u>bid</u>	bname	color	
Reserves	<u>sid</u>	<u>bid</u>	<u>day</u>	

- Q14: Find the average age of all sailors.

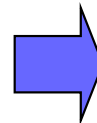
```
SELECT AVG (S.age)
FROM   Sailors S
```

- Q15: Find the average age of sailors with rating of 10

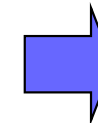
```
SELECT AVG (S.age)
FROM   Sailors S
WHERE  S.rating = 10
```

Sailors


```
SELECT age
FROM   Sailors S
WHERE  S.rating = 10
```



AVG(age)



25

Examples

sailors	<u>sid</u>	sname	rating	age
Boats	<u>bid</u>	bname	color	
Reserves	<u>sid</u>	<u>bid</u>	<u>day</u>	

- Q16: Find the name and age of the oldest sailor

```
SELECT S.sname, MAX (S.age)
FROM Sailors S
```

If the SELECT clause uses an aggregate operation, then it must use only aggregate operations unless the query contains a GROUP BY clause.

```
SELECT S.sname, S.age
FROM Sailors S
WHERE S.age =
      (SELECT MAX (S2.age)
       FROM Sailors S2)
```

Examples

sailors	<u>sid</u>	sname	rating	age
Boats	<u>bid</u>	bname	color	
Reserves	<u>sid</u>	<u>bid</u>	<u>day</u>	

- Q17: Count the number of Sailors.

```
SELECT COUNT (*)  
FROM Sailors S
```

- Q18: Count the number of different sailor names.

```
SELECT COUNT (DISTINCT S.name)  
FROM Sailors S
```

Examples

sailors	<u>sid</u>	sname	rating	age
Boats	<u>bid</u>	bname	color	
Reserves	<u>sid</u>	<u>bid</u>	<u>day</u>	

- Aggregate operations offer an alternative to the ANY and ALL constructs.
- Q19: Find the names of sailors who are older than the oldest sailor with a rating of 10.

```
SELECT S.name
FROM Sailors S
WHERE S.age > ( SELECT MAX (S2.age)
                FROM Sailors S2
                WHERE S2.rating = 10)
```

Group by and Having

- To write such queries, we need a major extension to the basic SQL query form, namely the **GROUP BY** clause.
- The extension also includes an optional **HAVING** clause that can be used to specify qualifications over groups.
- Q20: Find the age of the youngest sailor for each rating level.

```
SELECT  S.rating, MIN (S.age)
FROM    Sailors S
GROUP BY  S.rating
```

Group by

sailors	<u>sid</u>	sname	rating	age
Boats	<u>bid</u>	bname	color	
Reserves	<u>sid</u>	<u>bid</u>	<u>day</u>	

- Q20: Find the age of the youngest sailor for each rating level.

```
SELECT S.rating, MIN (S.age)
FROM Sailors S
GROUP BY S.rating
```

- For each group of tuples with the same rating, apply aggregate function min to age

sid	sname	rating	age
102	John	10	18
217	Mary	9	20
201	Athena	9	20
215	Joey	10	18
222	Rachel	5	25



sid	sname	rating	age
102	John	10	18
215	Joey	10	18
217	Mary	9	20
201	Athena	9	20
222	Rachel	5	25

Sailors table

rating	Min(age)
10	18
9	20
5	25



- The general format of **GROUP BY** and **HAVING**

SELECT	[DISTINCT] <i>target-list</i>
FROM	<i>relation-list</i>
WHERE	<i>qualification</i>
GROUP BY	<i>grouping-list</i>
HAVING	<i>group-qualification</i>

- The target-list contains
 - (i) attribute names
 - (ii) terms with aggregate operations (e.g., MIN (S.age)).
- The attribute list (i) **must be a subset of** grouping-list.
- Intuitively, each answer tuple corresponds to a group, and these attributes must have a **single value** per group.
- (A group is a set of tuples that have the same value for all attributes in grouping-list.)

Conceptual Evaluation

SELECT	[DISTINCT] <i>target-list</i>
FROM	<i>relation-list</i>
WHERE	<i>qualification</i>
GROUP BY	<i>grouping-list</i>
HAVING	<i>group-qualification</i>

- 1) Compute the cross-product of *relation-list*
- 2) Discard records that fail *qualification*
- 3) Partition the remaining records according to the value of attributes in *grouping-list*
- 4) Apply the *group-qualification* to eliminate groups that are not qualified
- 5) Delete attributes that are not specified by *target-list*, and *execute* aggregate operators.

Examples

sailors	<u>sid</u>	sname	rating	age
Boats	<u>bid</u>	bname	color	
Reserves	<u>sid</u>	<u>bid</u>	<u>day</u>	

- Q21: Find the age of the youngest sailor who are at least 18 years, for each rating with at least 2 such sailors.

```
SELECT S.rating, MIN (S.age)
FROM Sailors S
WHERE S.age >= 18
GROUP BY S.rating
HAVING COUNT (*) > 1
```

<u>sid</u>	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
71	zorba	10	16.0
64	horatio	7	35.0
29	brutus	1	33.0
58	rusty	10	35.0

- Only S.rating and S.age are mentioned in the SELECT, GROUP BY or HAVING clauses; other attributes are 'unnecessary'.
- 2nd column of result is unnamed. (Can use AS to name it)

rating	age
1	33.0
7	45.0
7	35.0
8	55.5
10	35.0

rating	
7	35.0

Answer relation

Examples

sailors	<u>sid</u>	sname	rating	age
Boats	<u>bid</u>	bname	color	
Reserves	<u>sid</u>	<u>bid</u>	<u>day</u>	

- Q22: For each red boat, find the number of reservations for this boat.

```
SELECT B.bid, COUNT (*) AS reservationcount
FROM Boats B, Reserves R
WHERE R.bid=B.bid
GROUP BY B.bid
HAVING B.color = 'red'
```



Only columns that appear in the GROUP BY clause can appear in the HAVING clause, unless they appear as arguments to an aggregate operator in the HAVING clause.

```
SELECT B.bid, COUNT (*) AS reservationcount
FROM Boats B, Reserves R
WHERE R.bid=B.bid AND B.color='red'
GROUP BY B.bid
```

Examples

sailors	<u>sid</u>	sname	rating	age
Boats	<u>bid</u>	bname	color	
Reserves	<u>sid</u>	<u>bid</u>	<u>day</u>	

- Q23: Find those ratings for which the average age is the minimum over all ratings

```
SELECT S.rating
FROM Sailors S
WHERE S.age = (SELECT MIN (AVG (S2.age))
               FROM Sailors S2
               GROUP BY S2.rating)
```



- Aggregate operations cannot be nested!**
 - This query will not work even if MIN(AVG(S2.age)), which is illegal, is allowed.
 - In the nested query, Sailors is partitioned into groups by rating, and the average age is computed for each rating value.
 - For each group, applying MIN to this average age value for the group will return the same value.

Examples

sailors	<u>sid</u>	sname	rating	age
Boats	<u>bid</u>	bname	color	
Reserves	<u>sid</u>	<u>bid</u>	<u>day</u>	

- Q23: Find those ratings for which the average age is the minimum over all ratings

```
SELECT Temp.rating, Temp.avgage
FROM (SELECT S.rating, AVG (S.age) AS avgage
      FROM Sailors S
      GROUP BY S.rating) AS Temp
WHERE Temp.avgage = (SELECT MIN (Temp.avgage)
                    FROM Temp)
```



- It essentially computes a temporary table containing the average age for each rating value and
- then finds the rating(s) for which this average age is the minimum.

Summary of SQL Queries

- A query in SQL can consist of up to six clauses, but only the first two, SELECT and FROM, are mandatory. The clauses are specified in the following order:

SELECT	<attribute list>
FROM	<table list>
[WHERE	<condition>]
[GROUP BY	<grouping attribute(s)>]
[HAVING	<group condition>]
[ORDER BY	<attribute list>]

Outline

- Introduction
- **Data Manipulation Language (DML)**
 - Simple SQL Queries
 - Nested SQL Queries
 - Aggregation and Grouping
 - Others, More Example: Relational Algebra and SQL
- Advanced Concepts (DDL)
 - Introduction to View
 - Introduction to Assertion/Trigger

NULL Value

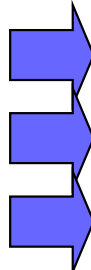
- Field value: UNKNOWN
 - For example: A new employee has not been assigned a supervisor yet.
- Field attribute inapplicable

Employee

<u>eno</u>	ename	supervisor_eno	spouse_name
------------	-------	----------------	-------------

eno	ename	supervisor_eno	spouse_name
123	Raymond	NULL	NULL
200	Peter	200	Mary
300	Mary	123	Peter

NULL Value



eno	ename	supervisor_eno		spouse_name
123	Raymond	NULL	UNKNOWN	NULL
200	Peter	300	FALSE	Mary
300	Mary	123	TRUE	Peter

```
SELECT *  
FROM employee  
WHERE supervisor_eno = 123
```

The condition following 'where' clause eliminates FALSE or unknown

eno	ename	supervisor_eno	spouse_name
300	Mary	123	Peter

Comparisons Involving NULL

- Each individual NULL value considered to be different from every other NULL value
- SQL uses a three-valued logic:
 - **TRUE**, **FALSE**, and **UNKNOWN**

Three-Valued Logic

Logical Connectives in Three-Valued Logic

(a)	AND	TRUE	FALSE	UNKNOWN
	TRUE	TRUE	FALSE	UNKNOWN
	FALSE	FALSE	FALSE	FALSE
	UNKNOWN	UNKNOWN	FALSE	UNKNOWN
(b)	OR	TRUE	FALSE	UNKNOWN
	TRUE	TRUE	TRUE	TRUE
	FALSE	TRUE	FALSE	UNKNOWN
	UNKNOWN	TRUE	UNKNOWN	UNKNOWN
(c)	NOT			
	TRUE	FALSE		
	FALSE	TRUE		
	UNKNOWN	UNKNOWN		

Comparisons Involving NULL

- Two rows are duplicates if matching columns are either equal or both NULL
- NULL is counted in COUNT(*)
- All other aggregate discard NULL values

OUTER JOIN

- In **NATURAL JOIN** and **EQUIJOIN**, tuples without a matching (or related) tuple are eliminated from the join result
 - Tuples with **NULL** in the join attributes are also eliminated
 - This amounts to loss of information
- A set of operations, called **OUTER** joins, can be used when we want to keep all the tuples in R, or all those in S, or all those in both relations in the result of the join, regardless of whether or not they have matching tuples in the other relation

OUTER JOIN

- The **left outer join** operation keeps every tuple in the **first** or **left relation** R; if no matching tuple is found in S, then the attributes of S in the join result are filled or “padded” with **NULL** values
- The **right outer join** keeps every tuple in the **second** or **right relation** S
- The **full outer join** keeps all tuples in **both** the left and the right relations when no matching tuples are found, padding them with **NULL** values as needed

Outer Joins

sailors	<u>sid</u>	sname	rating	age
Boats	<u>bid</u>	bname	color	
Reserves	<u>sid</u>	<u>bid</u>	<u>day</u>	

List (sid, bid) for sailors and boats they have reserved.

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

sid	bid	day
22	101	10/10/05
58	103	11/12/05

SELECT S.sid, R.bid

FROM Sailors S **NATURAL LEFT OUTER JOIN** Reserves R

All left relation
rows (Sailors)
appear in the
result

sid	bid
22	101
31	NULL
58	103

← Use of NULL

Outer Joins

sailors	<u>sid</u>	sname	rating	age
Boats	<u>bid</u>	bname	color	
Reserves	<u>sid</u>	<u>bid</u>	<u>day</u>	

List (sid, bid) for sailors and boats they have reserved.

sid	bid	day
22	101	10/10/05
58	103	11/12/05

sid	sname	rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

SELECT S.sid, R.bid

FROM Reserves R NATURAL RIGHT OUTER JOIN Sailors S

sid	bid
22	101
31	NULL
58	103

All right relation rows (Sailors) appear in the result

Outer Joins

sailors	<u>sid</u>	sname	rating	age
Boats	<u>bid</u>	bname	color	
Reserves	<u>sid</u>	<u>bid</u>	<u>day</u>	

SB

sid	bid
22	101
31	NULL
58	103

bid	bname	color
101	apollo	white
103	maria	blue
107	princess	pink

SELECT SB.sid, B.bid

FROM SB NATURAL FULL OUTER JOIN Boats B

sid	bid
22	101
31	NULL
58	103
NULL	107

Outline

- Introduction
- Data Manipulation Language (DML)
 - Simple SQL Queries
 - Nested SQL Queries
 - Aggregation and Grouping
 - Others, More Example: Relational Algebra and SQL
- **Advanced Concepts (DDL)**
 - **Introduction to View**
 - Introduction to Assertion/Trigger

Views in SQL

- A view is a “virtual” table that is derived from other tables
- Allows for limited update operations
- Since the table may not physically be stored
- Allows full query operations
- A convenience for expressing certain operations
- Provide a mechanism to hide certain data from the view of certain users

Views

List (sid, bid) for sailors and boats they have reserved.

```
CREATE VIEW SB (sid, bid)
AS SELECT S.sid, R.bid
FROM Reserves R NATURAL RIGHT OUTER JOIN Sailors S
```

sid	bid
22	101
31	NULL
58	103

View SB is a table which is **not** explicitly stored in the database.

A view can be used just like a base table.

Using a Virtual Table

- We can specify SQL queries on a newly create table

(view):

```
SELECT COUNT(*)  
FROM SB
```

- When no longer needed, a view can be dropped:

```
DROP VIEW SB;
```

Update Views

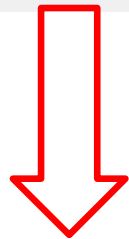
- Update on a single view without aggregate operations:
 - Update may map to an update on the underlying base table
- Views involving joins:
 - An update may map to an update on the underlying base relations
 - Not always possible

[1] Create a view StudentView2008 that contains only students born in 2008 .

```
CREATE VIEW StudentView2008
AS
SELECT *
FROM Student
WHERE year(birthday)=2008
```

[2] In StudentView2008, change the name of the student whose student number is 1600004 to 'Tom'.

```
UPDATE StudentView2008
SET studentName='Tom'
WHERE studentNo='1600004'
```



Convert to update the underlying base table

```
UPDATE Student
SET studentName='Tom'
WHERE year(birthday)=2008 AND studentNo='1600004'
```


Summary

- SQL was an important factor in the early acceptance of the relational model; more natural than earlier, procedural query languages.
- Relationally complete; in fact, significantly more expressive power than relational algebra.

Relational Algebra and SQL

- Exercise 1: Consider the following schema:
 - Suppliers(sid: integer, sname: string, address: string)
 - Parts(pid: integer, pname: string, color: string)
 - Catalog(sid: integer, pid: integer, cost: real)
- The key fields are underlined, and the domain of each field is listed after the field name. Therefore sid is the key for Suppliers, pid is the key for Parts, and sid and pid together form the key for Catalog.

Example

Suppliers

<u>sid</u>	sname	address
------------	-------	---------

Parts

<u>pid</u>	pname	color
------------	-------	-------

Catalog

<u>sid</u>	<u>pid</u>	cost
------------	------------	------

- Q1.1: Find the names of suppliers who supply some green part.

?

?

Example

Suppliers

<u>sid</u>	sname	address
------------	-------	---------

Parts

<u>pid</u>	pname	color
------------	-------	-------

Catalog

<u>sid</u>	<u>pid</u>	cost
------------	------------	------

- Q1.2: Find the sids of suppliers who supply some red or green part.

?

?

Example

Suppliers	<u>sid</u>	sname	address
Parts	<u>pid</u>	pname	color
Catalog	<u>sid</u>	<u>pid</u>	cost

- Q1.3: Find the sids of suppliers who supply every part.

(Solution 1)

?

?

Example

Suppliers

<u>sid</u>	sname	address
------------	-------	---------

Parts

<u>pid</u>	pname	color
------------	-------	-------

Catalog

<u>sid</u>	<u>pid</u>	cost
------------	------------	------

- Q1.3: Find the sids of suppliers who supply every part.

(Solution 2)

?

?

END